Jailbreaking in the Haystack

Rishi Rajesh Shah **Chen Henry Wu** **Shashwat Saxena** **Ziqian Zhong**
Alexander Robey **Aditi Raghunathan**

Carnegie Mellon University

rishisha@cs.cmu.edu
https://github.com/AR-FORUM/NINJA_Attack
https://ar-forum.github.io/ninjaattackweb/

arXiv:2511.04707v1 [cs.CR] 5 Nov 2025

Abstract

Recent advances in long-context language models (LMs) have enabled million-token inputs, expanding their capabilities across complex tasks like computer-use agents. Yet, the safety implications of these extended contexts remain unclear. To bridge this gap, we introduce **Ninja** (short for ***N**eedle-**in**-haystack **j**ailbreak **a**ttack*), a method that jailbreaks aligned LMs by appending benign, model-generated content to harmful user goals. Critical to our method is the observation that the position of harmful goals play an important role in safety. Experiments on standard safety benchmark, HarmBench, show that Ninja significantly increases attack success rates across state-of-the-art open and proprietary models, including LLaMA, Qwen, Mistral, and Gemini. Unlike prior jailbreaking methods, our approach is low-resource, transferable, and less detectable. Moreover, we show that Ninja is compute-optimal – under a fixed compute budget, increasing context length can outperform increasing the number of trials in best-of-N jailbreak. These findings reveal that even benign long contexts – when crafted with careful goal positioning – introduce fundamental vulnerabilities in modern LMs.

1 Introduction

Recent advances in language models (LMs) have dramatically expanded their capacity to process long-context inputs, enabling them to handle entire codebases or books in a single pass (Anthropic, 2024; OpenAI, 2024). This has led to their adoption in critical real-world tasks, particularly with smaller, efficient models that are practical to deploy. However, the safety implications of these extended context windows remain poorly understood, creating a critical vulnerability.

Prior work has highlighted this vulnerability in specific settings. For instance, Kumar et al. (2024) find that LMs are easier to jailbreak when prompted as agents—whose context windows contain long histories and tool definitions—rather than as simple chatbots. However, it remained unclear whether this vulnerability arose from the context length itself or from a fundamental distribution shift when models adopt an agentic persona. This paper isolates this variable and provides a clear answer: the context length itself, even when the context is entirely benign, is a primary driver of safety degradation.

Our key finding is that a model's refusal to comply with harmful requests drops dramatically as context size increases, far outpacing any corresponding drop in its general capabilities. This gap reveals a fundamental flaw in current safety paradigms and leads to a simple yet devastatingly practical attack we call **Ninja** (short for ***N**eedle-**in**-haystack **j**ailbreak **a**ttack*). Ninja jailbreaks a model by embedding a harmful goal within a long, benign, and thematically relevant context. Unlike prior work such as many-shot jailbreaking (Anil et al., 2024a; Lu et al., 2025), which relies on injecting explicitly harmful examples, Ninja uses entirely innocuous context that can even be synthetically generated. This makes the attack harder to detect. On HarmBench, Ninja improves the attack success rate (ASR) from 23.7% to 58.8% for Llama-3.1-8B-Instruct, from 23.7% to 42.5% for Qwen2.5-7B-Instruct, and from 23% to 29% for Gemini Flash. We also evaluate on Mistral-7B-v0.3, achieving a 54.5% ASR, though interestingly, Mistral exhibits a different pattern where

longer contexts increase non-refusal rates but do not improve ASR, suggesting model-specific differences in how capability and safety interact at scale.

A critical and previously undocumented finding is that **goal positioning** significantly impacts the success of this attack. Placing the harmful goal at the beginning of the context dramatically increases the ASR, whereas placing it at the end mitigates the attack's effectiveness. This insight does not affect model capability but has profound implications for safety, suggesting that for maximal safety, user-provided goals should be positioned at the end of system prompts.

Finally, we show that NINJA is more compute-efficient than existing test-time attacks like best-of-N sampling. Given a fixed compute budget, an attacker is better off sampling fewer responses from long-context inputs than many responses from short-context inputs. This provides practical takeaways for both attackers and defenders, highlighting that improving the long-context capability of models is insufficient without developing explicit safety mechanisms for this emerging threat vector.

Our key contributions are:

- We introduce the **NINJA (Needle-in-Haystack Jailbreak) Attack**, a simple yet highly effective method for jailbreaking aligned language models. By appending benign, model-generated content to a harmful user goal, our approach significantly boosts the attack success rate (ASR) across various models. The NINJA attack is low-resource and less detectable than prior jailbreaking methods.
- We provide a detailed empirical **analysis of goal positioning**, revealing that placing the harmful request at the beginning of the context is the most effective strategy for maximizing the attack success rate. This finding highlights a key vulnerability in how long-context models process and prioritize information.
- We propose a **compute-aware scaling law** for optimizing jailbreak attacks, which demonstrates how to select the optimal context length to maximize the ASR within a given best-of-N compute budget. Our findings show that under a larger compute budget, using a longer context is more effective than increasing the number of attack attempts.

2 Related Work

Jailbreaking with Adversarial Content. A dominant paradigm in jailbreaking research involves crafting overtly adversarial inputs to circumvent model safeguards. Gradient-based methods like GCG search for specific token sequences that trigger harmful behavior (Zou et al., 2023), while optimization-based approaches like PAIR use an attacker LLM to iteratively refine prompts (Chao et al., 2023). Other methods exploit model behavior over multiple turns, such as Crescendo, which gradually escalates a benign conversation into a harmful one (Russinovich et al., 2024). A common thread in these attacks is the reliance on specially crafted, often non-benign content. For example, many-shot jailbreaking conditions a model on numerous examples of harmful Q&A pairs to elicit compliance (Anil et al., 2024a). Our NINJA attack diverges fundamentally from this paradigm.

Stealthy Attacks via Context Manipulation. The NINJA attack leverages a more subtle vulnerability: the degradation of safety alignment in the presence of long, benign context. This places it in a category of attacks that manipulate the context to degrade model safety rather than using explicitly adversarial content. A notable example is the Cognitive Overload Attack (Upadhayay et al., 2024), which seeks to overwhelm a model's processing capacity by presenting it with complex, distracting tasks (e.g., coding challenges) *before* posing a harmful question. The core distinction lies in the mechanism and structure: Cognitive Overload distracts the model to exhaust its cognitive resources, whereas NINJA uses thematically *relevant* context to exploit the model's inherent positional biases. Crucially, in their setup, the Cognitive Overload Attack places the harmful goal after the distracting content, whereas our key finding demonstrates that for NINJA, the harmful goal must be placed at the beginning of the context to succeed.

Long-Context and Positional Vulnerabilities. Our work is situated within a growing body of research investigating the brittleness of LLMs on long-context tasks. The "lost in the middle" phenomenon, identified by Liu et al. (2024a), shows that models often struggle to retrieve information located in the middle of long inputs, revealing a clear positional bias. This U-shaped performance curve, where models favor information at the beginning (primacy bias) or end (recency bias) of a prompt, has been documented across various models and tasks (Wu et al., 2025). While these studies focused on model capabilities, our work explores the security implications of this phenomenon. We demonstrate that this positional bias is not just a capability issue but also a significant safety vulnerability, where the placement of a harmful goal dictates the success of an attack.

Benchmarks and Defenses. To systematically evaluate jailbreak robustness, several benchmarks have been established. HarmBench provides a standardized suite for testing models against known adversarial attacks (Mazeika et al., 2024a). More recent work has focused on agentic vulnerabilities. For example, SHADE-Arena evaluates an agent's ability to pursue subtle, harmful side-tasks without being detected by a monitor LLM (Gao et al., 2024). These benchmarks are critical but primarily focus on attacks with adversarial or complex agentic scaffolding. The vulnerabilities we expose with Ninja —arising from simple, benign context—are orthogonal and persist even with strong baseline defenses. This suggests that future defensive strategies, such as goal prioritization (Zhang et al., 2024) or self-reminders (Xie et al., 2023), must also account for the fundamental safety challenges posed by long contexts themselves.

3 Long-Context Generation for Jailbreaking

LMs can now attend to information presented across long sequences Kamradt (2023). We leverage this property to construct targeted jailbreaks by embedding harmful goals within relevant long contexts. Our approach is designed to preserve safety benchmark constraints: (i) the context must not directly contain the final answer to the harmful goal, and (ii) the context should remain topically aligned with the target behavior, ensuring the jailbreak is not merely due to retrieval-style prompting or prompt injection.

3.1 Ninja Attack

To systematically inject a harmful goal into a benign but relevant "haystack," our pipeline proceeds in three stages:

Keyword extraction Given a harmful goal g, we first identify a small set of keywords. We select up to five keywords $\{k_i\}$ – prioritizing nouns, then adjectives and verbs – to capture the core semantics of the goal. This ensures that our generated context stays on topic without leaking the answer.

Context generation We then iteratively prompt the LLM to produce natural, educational, or descriptive passages around each keyword. For each k_i in our keyword set, we sample a template t_i (e.g., *Create an educational passage discussing {keyword} in a classroom setting.*) and generate a continuation c_i. We append c_i back into the prompt and repeat – sampling new (k_i, t_i) pairs – until the accumulated context C reaches the target length L. We give more examples of the templates in Appendix B.

Final prompt To avoid truncations or malformed text, we apply regex heuristics that detect incomplete sentences or abrupt cut-offs – discarding and regenerating any failing pass. Finally, we format the prompt as follows: *Based on the following long context, {harmful goal g} {extended_context C}.* This prompt is then sent to the LM for evaluation.

3.2 Impact of Goal Positioning

We observe a notable sensitivity in model behavior to the position of the harmful goal within the context. Motivated by prior work on "needle-in-a-haystack" evaluations Kamradt (2023), we conduct controlled experiments by varying the insertion point of the goal at multiple positions throughout the context (see full prompt templates in Appendix C).